# Informe

*Parcial práctico de Ingeniería de Software. Integrantes:*

## 1. Problema, actores y alcance

**Problema.** Un colectivo de escritores sin formación en programación prepara una ficción interactiva para un festival universitario. Hoy, agregar una escena o cambiar una decisión obligaría a modificar el programa, y un error en el contenido podría cerrar la aplicación frente al lector. Se necesita un motor que interprete historias desde archivos, condicione las opciones según lo ocurrido y avise con claridad cuando un archivo tiene errores.

**Actores.**

| Actor | Rol en el sistema |
|---|---|
| Escritora principal | Edita los archivos de historia; no programa. |
| Diseñador de la experiencia | Define condiciones y consecuencias de las opciones. |
| Coordinadora del festival | Revisa los diagnósticos de errores antes de publicar. |
| Lector | Juega la historia y elige opciones. |
| Equipo desarrollador | Construye y mantiene el motor. |

**Alcance incluido.** Carga de historias desde archivo JSON, validación previa, sesión de juego con variables booleanas, opciones condicionadas, efectos, finales, bloqueo controlado, reinicio, interfaz local, deshacer de un solo nivel (entra por la solicitud de cambio, RF19 a RF23) y una segunda historia que funciona sin tocar el motor.

**Fuera de alcance.** Cuentas, base de datos, guardado entre ejecuciones, internet, editor visual, audio, imágenes y lenguaje de expresiones.

## 2. Dos riesgos con su mitigación

| Riesgo | Prob. / Impacto | Mitigación |
|---|---|---|
| **R1.** El cliente decide más adelante sobre el retroceso y obliga a rediseñar la sesión. | Media / Alto | Todo el estado de la partida vive en un único objeto Sesión, de modo que agregar historial afectaría un solo componente. Se documenta el conflicto y el acuerdo provisional (sección 6). |
| **R2.** La solicitud de cambio de la hora 6 llega cuando el motor ya tiene lógica atada a la historia o la primera versión está incompleta. | Alta / Alto | Motor dirigido por datos, sin IDs de la historia en el código; segunda historia como prueba de desacople; guardar versión inicial en la hora 6 y reservar 1 h 30 min para cambio y pruebas. |

## 3. Decisión de responsabilidad profesional

**Decisión: dependencias, autoría y uso de IA con trazabilidad.** Solo usamos Python y su biblioteca estándar (`json`, `copy`, `sys`, `unittest`). No hay bibliotecas externas. La historia y las rutas del cliente se conservan tal como se entregaron; solo se amplía la redacción si es compatible con las reglas. Declaramos en el informe qué fragmentos, tutoriales o herramientas de IA se usaron, para qué sirvieron y cómo se verificó el resultado (pruebas automatizadas y revisión cruzada entre integrantes), distinguiendo el trabajo propio del material adaptado.

## 4. Requisitos funcionales

RF01 a RF18 cubren las necesidades N01 a N06. RF19 a RF23 provienen de la solicitud de cambio (deshacer de un solo nivel).

| ID | Nec. | Requisito (comportamiento observable) | Criterio de aceptación |
|---|---|---|---|
| RF01 | N01 | El sistema carga una historia desde un archivo que indica el usuario. | Dado un archivo válido, se abre una sesión. Dos archivos distintos cargan dos historias distintas sin cambiar el código. |
| RF02 | N01 | Al iniciar una partida, la escena actual es la indicada en `inicio` y las variables toman los valores de `variables`. | En la historia de referencia, la partida empieza en E1 con `conoce_origen` y `confia_en_la_voz` en falso. |
| RF03 | N02 | El sistema muestra el texto de la escena actual. | Al iniciar se muestra el texto de E1. |

| ID | Nec. | Requisito (comportamiento observable) | Criterio de aceptación |
|---|---|---|---|
| RF04 | N02 | Solo se muestran las opciones cuyas condiciones se cumplen todas; una opción sin condiciones siempre se muestra. | CA01: en E3 con `conoce_origen` verdadero se ven Activar con código y Cortar comunicación. Con falso, Activar sin verificar y Cortar comunicación: exactamente dos opciones. |
| RF05 | N03 | Al elegir una opción habilitada, se aplican sus efectos y la sesión pasa al destino. | Elegir Investigar archivo en E2 deja la sesión en E3 con `conoce_origen` verdadero. |
| RF06 | N03 | Si se elige una opción inexistente, el sistema informa el error y no cambia escena ni variables. | Con el ID `xyz`, el estado es idéntico antes y después. |
| RF07 | N03 | Si se elige una opción no habilitada, incluso enviando su ID directamente al motor, el sistema la rechaza sin cambiar el estado. | CA02: en E3 con `conoce_origen` falso, Activar con código se rechaza y la sesión sigue en E3. |
| RF08 | N03 | Si la entrada del lector es vacía, no numérica o fuera de rango, la interfaz informa el error y repite la escena sin alterar la partida. | Entradas vacía, `abc` y `9` no modifican el estado. |
| RF09 | N04 | Los valores de las variables se conservan durante toda la partida; un mapa de efectos vacío las deja intactas. | Tras Continuar sin investigar las variables siguen iguales. Tras Investigar, `conoce_origen` sigue verdadero en E3. |
| RF10 | N04 | Condiciones y efectos se evalúan únicamente con los datos del archivo, sin reglas específicas de una escena. | La segunda historia, con otros IDs y variables, funciona sin editar el motor. |
| RF11 | N05 | Al llegar a una escena final, el sistema muestra su texto, indica que es un desenlace y ofrece reiniciar o salir. | Las rutas hacia E4, E5 y E6 terminan con el aviso de final y sin opciones de avance. |
| RF12 | N05 | Reiniciar restaura la escena inicial y las variables iniciales, sin información de la partida anterior. | Tras llegar a E5 y reiniciar, la escena es E1 y ambas variables están en falso. |
| RF13 | N05 | Si una escena no final no tiene opciones habilitadas, el sistema informa el bloqueo y permite reiniciar. | Un contenido de prueba con una escena sin salida muestra el aviso y ofrece reiniciar, sin cierre del programa. |
| RF14 | N06 | Antes de iniciar, el sistema rechaza un archivo ilegible, con estructura incorrecta o sin campos obligatorios, indicando causa y campo. | Un JSON mal formado y otro sin `escenas` producen mensajes distintos que nombran el problema. |
| RF15 | N06 | El sistema rechaza una escena inicial inexistente, IDs de escena repetidos, IDs de opción repetidos dentro de una escena y destinos inexistentes, indicando el ID involucrado. | CA03: un destino `E99` inexistente produce un mensaje que nombra la opción, la escena y `E99`, y no se abre sesión. |
| RF16 | N06 | El sistema rechaza condiciones o efectos que usan variables no declaradas o valores no booleanos, indicando variable y opción. | Un efecto sobre `variable_x` no declarada o con valor `"si"` se rechaza con mensaje. |
| RF17 | N06 | El sistema rechaza escenas finales con opciones y escenas no finales sin opciones. | Una escena final con una opción y una no final con lista vacía producen errores distintos. |
| RF18 | N06 | Un archivo inválido nunca genera una sesión jugable ni termina el programa de forma inesperada. | Con cualquier archivo de RF14 a RF17, no existe sesión y el programa responde con mensaje y código de salida controlado. |
| RF19 | Cambio | Deshacer restaura la escena y todas las variables anteriores a la última elección válida. | Desde E2, investigar lleva a E3 con `conoce_origen` verdadero; deshacer vuelve a E2 con `conoce_origen` falso. |
| RF20 | Cambio | Solo hay un nivel de deshacer: después de deshacer no se puede deshacer otra vez hasta hacer una nueva elección válida. | Un segundo deshacer no cambia nada. Si luego se elige Continuar sin investigar, E3 muestra Activar sin verificar y Cortar comunicación. |
| RF21 | Cambio | Al comenzar una partida no hay retroceso disponible, y reiniciar borra el historial. | En una partida nueva o recién reiniciada, deshacer no cambia nada. |
| RF22 | Cambio | Una opción inexistente o no habilitada no modifica la sesión ni reemplaza el historial. | Tras una elección válida y una inválida, deshacer restaura el estado anterior a la válida. |
| RF23 | Cambio | Se puede deshacer desde E4, E5 o E6: se vuelve a E3 con las variables previas a esa elección. | Desde cada final, deshacer deja la sesión en E3 con las variables que tenía antes de elegir. |

## 5. Tres preguntas y respuestas (supuestos)

Como no hay respuesta del cliente durante la ventana, cada respuesta queda registrada como supuesto compatible con los requisitos explícitos.

| # | Pregunta al cliente | Respuesta o supuesto adoptado |
|---|---|---|
| P1 | ¿El lector elige escribiendo el número de la opción o el identificador? | **Supuesto:** la interfaz numera las opciones visibles (1, 2, ...) y las traduce al ID interno; el motor acepta IDs, lo que permite probar CA02. |
| P2 | Ante un archivo con varios errores, ¿se informa solo el primero o todos? | **Supuesto:** se informan todos en una lista, para que la escritora corrija el archivo de una sola vez (necesidad de la coordinadora: saber qué corregir). |
| P3 | ¿Se permiten historias donde una escena se pueda visitar más de una vez (ciclos)? ¿Debe verificarse que todo final sea alcanzable? | **Supuesto:** los ciclos están permitidos y no se analiza la alcanzabilidad de todos los caminos, como indica la guía; solo se informa el bloqueo en tiempo de juego (RF13). |

## 6. Conflicto del retroceso

**Conflicto.** La escritora quiere permitir retroceder siempre. El diseñador quiere decisiones irreversibles, porque las decisiones deben tener consecuencias. Ambas posturas son legítimas y no pueden satisfacerse a la vez sin una regla adicional.

**Pregunta de aclaración al cliente.** ¿El retroceso debe estar permitido en todas las decisiones, solo en las que la escritora marque como reversibles, o en ninguna? Si se retrocede, ¿deben restaurarse también las variables al valor que tenían antes de la decisión?

**Acuerdo provisional (versión inicial).** No se implementa retroceso. Todas las decisiones son irreversibles y la única forma de volver al inicio es reiniciar (RF12). Este acuerdo es temporal y no presupone la decisión del cliente. Si pide retroceso, se tratará como un cambio con análisis de impacto, y el estado de la partida está encapsulado en un único objeto Sesión para acotar ese impacto (ver R1). La solicitud de cambio reemplazó el acuerdo provisional (RF19 a RF23).

## 7. Proceso elegido y alternativa

**Proceso elegido: incremental e iterativo con prácticas ágiles ligeras** (backlog, tablero, entregas pequeñas con pruebas). **Alternativa comparada: cascada (secuencial).**

| Criterio | Incremental / ágil (elegido) | Cascada (alternativa) |
|---|---|---|
| Cambio en la hora 6 | Se absorbe replanificando el backlog; la versión inicial ya está guardada. | Obliga a reabrir requisitos y diseño ya cerrados. |
| Ambigüedades del cliente (retroceso) | Se resuelven con supuestos y se corrigen en el siguiente incremento. | Exigen resolverse antes de avanzar. |
| Duración y tamaño | Adecuado a 12 h y dos personas: cada incremento deja software ejecutable y probado. | La documentación completa previa consume tiempo y retrasa el primer software ejecutable. |
| Riesgo | Se detecta temprano con pruebas por incremento. | Los defectos aparecen al final, en la integración. |

Incrementos previstos: **I1** núcleo del motor con la historia de referencia (carga, condiciones, efectos, finales); **I2** validación de contenido; **I3** interfaz de consola, reinicio y segunda historia; **I4** solicitud de cambio, pruebas finales y entrega.

**Cómo se realizan las cuatro actividades.**

| Actividad | En esta entrega |
|---|---|
| Especificación | Necesidades N01 a N06, RF con criterios de aceptación, 3 RNF medibles, preguntas y supuestos, revisión cruzada entre integrantes. |
| Desarrollo | Incrementos I1 a I3 con commits de ambos integrantes; separación en clases de contenido (JSON), reglas (`Evaluador`, `Sesion`) e interfaz (`InterfazConsola`). |
| Validación | Pruebas automatizadas con `unittest` (U1 a U5 e I1 en cada incremento, U6 tras el cambio); recorrido manual de los tres finales; verificación local de los RNF. |
| Evolución | Versión de partida marcada con el tag `v1-antes-del-cambio`, análisis de impacto de la solicitud de cambio, actualización de requisitos, modelos, pruebas y matriz de trazabilidad. |

## 8. Tres requisitos no funcionales medibles

| ID | Atributo | Requisito medible | Procedimiento de verificación | Evidencia |
|---|---|---|---|---|
| RNF01 | Modificabilidad del contenido | Agregar una escena nueva con dos opciones a una historia existente exige modificar solo el archivo de historia (0 archivos de código cambiados) y toma 10 minutos o menos siguiendo el README. | Copiar la historia, agregar la escena, ejecutar el motor y comprobar con `git diff --stat` que solo cambió el JSON. Cronometrar el proceso. | Prueba que carga la segunda historia sin tocar el motor, salida del `git diff --stat` y tiempo medido: [resultado y ruta del archivo de evidencia]. |
| RNF02 | Claridad de errores | En cada una de las 9 categorías de error de contenido, el mensaje indica causa y campo o ID involucrado (9 de 9) y ninguna termina con una excepción no controlada. | Prueba parametrizada con un archivo defectuoso por categoría. Las aserciones verifican que el mensaje contiene el ID o campo esperado y que el programa devuelve un código de salida controlado. | Registro de ejecución de pruebas: [ruta del archivo de evidencia]. |
| RNF03 | Confiabilidad | En 1.000 secuencias aleatorias de entradas válidas e inválidas (incluidas vacías) sobre ambas historias hay 0 excepciones no controladas, y las entradas inválidas no cambian escena ni variables. | Script con semilla fija que genera las secuencias, ejecuta el motor y compara el estado antes y después de cada entrada inválida. | Script de la verificación y resultado: [resultado y ruta del archivo de evidencia]. |

## 9. Prácticas ágiles, definición de terminado y aprendizaje

**Prácticas aplicadas.**

1. **Backlog priorizado y tablero Kanban con responsables.** Columnas: Por hacer, En curso, En revisión y Hecho, con un máximo de dos tarjetas en curso por persona. Se toman capturas del tablero al inicio y al final. Sirve para repartir el trabajo y ver el avance en una ventana de 12 horas.
2. **Revisión por pares con commits pequeños.** Ningún cambio llega a la rama principal sin que lo revise el otro integrante. Esto reduce defectos y deja un historial de Git con ambos autores.

**Alternativa descartada: Scrum con sprints y reuniones diarias.** Con dos personas y 12 horas, el costo de las ceremonias supera su beneficio. Se reemplaza por tres puntos de sincronización: al inicio, en la hora 6 con la solicitud de cambio y antes de entregar.

**Definición de terminado.** Una tarjeta pasa a Hecho solo si se cumple todo lo siguiente:

- El código está en la rama principal y lo revisó el otro integrante.
- Las pruebas automatizadas relacionadas pasan.
- El requisito está vinculado a su evidencia en la matriz de trazabilidad.
- El README se actualizó si cambió la instalación, la ejecución o el formato del contenido.

**Aprendizaje y acción de mejora.** El primer validador solo cubría 3 de las 9 categorías de error; lo detectó la revisión del código con ayuda de Claude. Acción de mejora: escribir primero las pruebas U4 parametrizadas.

## 10. Dos decisiones de arquitectura y separación

La arquitectura tiene cuatro componentes: **contenido** (archivos de historia), **cargador y validador**, **motor** (sesión y evaluación de condiciones) e **interfaz** (consola). El diagrama de componentes está en el Anexo (figuras A.6 y B.6).

**AD1. Motor dirigido por datos.**

- **Contexto:** la escritora no programa, el RNF01 exige modificar contenido sin tocar código y habrá un cambio en la hora 6.
- **Alternativas:** (a) codificar la historia en el programa; (b) crear un lenguaje de expresiones, que está fuera del alcance; (c) describir escenas, condiciones y efectos en JSON y que el motor los interprete.
- **Elección:** (c).
- **Consecuencias:** se puede cargar una segunda historia sin cambiar el motor. A cambio, la validación se vuelve obligatoria y la expresividad queda limitada a igualdades booleanas.

**AD2. Validador independiente del motor.**

- **Contexto:** la coordinadora necesita saber qué corregir antes de que alguien juegue (N06).

- **Alternativas:** (a) validar durante la partida; (b) validar dentro del motor; (c) un validador separado que devuelve la lista de errores y que se ejecuta antes de abrir la sesión.
- **Elección:** (c).
- **Consecuencias:** el motor solo recibe historias válidas y los errores se prueban por separado. El motor mantiene aun así la defensa de rechazar opciones no habilitadas (RF07), lo que repite parte de la comprobación.

**Justificación de la separación contenido, reglas e interfaz.** Cada capa cambia por una razón distinta: el contenido cambia cuando los escritores editan la historia, las reglas cuando cambian los requisitos y la interfaz cuando cambia la plataforma. El motor no imprime ni lee teclado, solo recibe órdenes y devuelve resultados, así que una interfaz web podría reemplazar la consola sin tocarlo. La separación se demuestra con la segunda historia, que funciona con el mismo motor, y con las pruebas del motor que corren sin interfaz.

## 11. Decisión de diseño detallado, bibliotecas y licencias

**Decisión de diseño detallado.** Solo la Sesión modifica el estado de la partida (escena actual y variables). El evaluador de condiciones es una clase sin estado (`Evaluador.cumple`) que recibe las condiciones y las variables y devuelve verdadero o falso. La interfaz usa tres operaciones del motor (`opciones_habilitadas`, `elegir(id)` y `reiniciar`), que devuelven resultados y no imprimen. Así el acoplamiento es bajo, cada pieza se prueba aislada y una opción rechazada no puede dejar el estado a medias (ver Anexo, figura A.3).

**Bibliotecas y licencias.**

| Biblioteca | Versión | Licencia | Para qué se usa |
|---|---|---|---|
| Python (biblioteca estándar: `json`, `copy`, `sys`, `unittest`) | [versión de Python] | PSF | Leer los archivos de historia (JSON) y correr las pruebas (`unittest`) |

## 12. Análisis de impacto del cambio

**Versión de partida:** tag `v1-antes-del-cambio`.

| | | | |
|---|---|---|---|
| Requisitos (RF/RNF) | Sí | Se agregan RF19 a RF23. | Sección 4 de este informe |
| Modelos | Sí | Casos de uso, clases, secuencia y estados. | Anexo, figuras B.2 a B.5 |
| Componentes y archivos de código | Solo Sesión | Cambia únicamente `Sesion` en `main.py`; los demás componentes no cambian. [detalle exacto de lo modificado] | `main.py`, clase `Sesion` |
| Pruebas | Sí | U6 nueva; U5 e I1 extendidas. | [archivos de pruebas] |
| Backlog y tablero | Sí | Se agregan los ítems del cambio. | Anexo, sección D |
| Matriz de trazabilidad | Sí | Se agregan RF19 a RF23 con modelos, código y pruebas. | Anexo, sección C |

## 13. Aportes por integrante, fuentes y herramientas de IA

**Aportes por integrante.**

| Integrante | Aportes | Evidencia en el historial |
|---|---|---|
| Kiefer Monterroza | [aportes concretos: archivos, clases o tareas] | V1- V2 |
| Jesús Vergara | [aportes concretos: archivos, clases o tareas] | V1-V2 |

**Fuentes y herramientas de apoyo.**

| Herramienta o fuente | Para qué se usó | Cómo se verificó |
|---|---|---|
| Claude (Anthropic), asistente de IA | Borradores de redacción del informe y de los diagramas, y revisión y corrección de `main.py`. | Revisión por ambos integrantes contra la guía del parcial; se verificó ejecutando el programa y las pruebas. |